Report on

SMARTMAIL ASSISTANT

Submitted to:

University Institute of Computing

Submitted by:

Rishi Soni



Chandigarh University, Mohali

BONAFIDE CERTIFICATE

Certified that this project report on **SMARTMAIL ASSISTANT** is the bonafide work of **Rishi Soni**, carried out as a project under the supervision of the undersigned.

SIGNATURE	SIGNATURE
SUPERVISOR	HEAD OF THE DEPARTMENT
Name: _____	University Institute of Computing
Designation: _____	Chandigarh University

Submitted for the project viva-voce examination held on: _____

INTERNAL EXAMINER	EXTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER 1 **7**

1.1 Need Identification 7

1.2 Identification of Problem 7

1.3 Identification of Tasks 7

1.4 Timeline 8

1.5 Organization of the Report 8

CHAPTER 2 **9**

2.1 Timeline of the Reported Problem 9

2.2 Proposed Solution 9

2.3 Bibliometric Analysis 10

2.4 Review Summary 10

2.5 Problem Definition 11

2.6 Goals / Objectives 11

2.7 Why Use These Technologies Over Other Options 11

CHAPTER 3 **12**

3.1 Evaluation and Selection of Specifications / Features 12

3.2 Design Constraints 12

3.3 Design Flow 13

3.4 Creation of SmartMail Assistant 13

3.5 Logo Designing and Naming 15

3.6 Implementation Plan / Methodology 15

CHAPTER 4 **16**

4.1 Implementation of Solution 16

4.1.1 Report Preparation 16

4.1.2 Project Management and Communication 16

4.1.3 Testing / Characterization / Interpretation / Data Validation 17

CHAPTER 5 **18**

5.1 Conclusion 18

5.2 Future Work 18

APPENDICES

A. Source code and system walkthrough 15-24

B. Operations and user guide 25-32

C. Reserved application screenshots 33-37

D. API, data handling, and security 38-46

E. Extended validation plan 47-58

F. Maintenance and handover 59-64

ABSTRACT

SmartMail Assistant is an AI-assisted email reply application that accepts the text of an existing email and returns a draft response in a selected tone. The project combines a React and Material UI interface, a Spring Boot REST service, Google's Gemini generative AI API, and a Chrome Manifest V3 content script intended to place an AI Reply action inside Gmail. The service receives an email body and tone, builds a natural-language prompt, sends the request to Gemini, extracts the first returned text part, and returns that text to the client. The React interface also validates blank input, shows a loading state and error message, and allows the result to be copied. The Gmail content script reads visible message text and inserts the generated response into the compose box. The design aims to reduce repetitive drafting effort while keeping the generated response under the user's review before sending.

The implementation is a prototype rather than a production-ready mail service. Review of the supplied source identifies important limitations: the browser extension and React interface are separate client implementations; the extension uses fixed tone and local API address values; Gmail DOM selectors are brittle; server-side request validation and model error handling are limited; and there is no evidence in the supplied repository of completed end-to-end tests or deployment configuration. The report describes these boundaries explicitly and proposes validation, privacy, reliability, and deployment improvements.

Keywords: email assistance, generative AI, Gemini API, Spring Boot, React, Chrome extension

LIST OF FIGURES

Fig. 1. Graphical abstract of SmartMail Assistant

Fig. 2. High-level system architecture

Fig. 3. Request and response sequence

Fig. 4. User workflow for generating a reply

Fig. 5. Software component view

Fig. C.1. Application home screenshot placeholder

Fig. C.2. Input and tone selection screenshot placeholder

Fig. C.3. Loading state screenshot placeholder

Fig. C.4. Generated reply screenshot placeholder

Fig. C.5. Gmail draft insertion screenshot placeholder

GRAPHICAL ABSTRACT

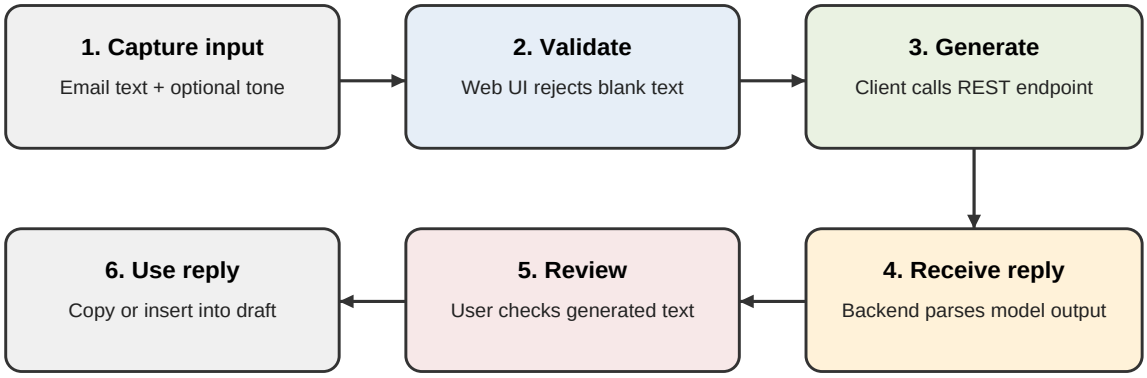


Fig. 1. Graphical abstract of the SmartMail Assistant workflow

The user supplies the original message, chooses or accepts a tone, reviews the generated draft, and decides whether to copy or insert it. The application does not send email automatically.

CHAPTER 1

INTRODUCTION

1.1 Need Identification

Email remains a routine channel for academic, business, and personal coordination. Many messages call for predictable responses such as acknowledging a request, confirming a meeting, asking for clarification, or providing a short update. Repeatedly composing these responses consumes time and can make tone and clarity inconsistent. A drafting assistant can reduce this repetitive effort by proposing text from the message context, while leaving the sender responsible for checking accuracy and deciding whether to send it.

SmartMail Assistant addresses this narrow productivity need. It is designed to work through a small web interface or a Gmail content script. The user provides the message content; a backend prepares a prompt for a generative model; and the resulting text is returned as a draft. A tone choice can guide the wording. The intended benefit is faster first-draft creation, not autonomous correspondence.

Generative output can be fluent while still being inaccurate, incomplete, or inappropriate. For that reason, the interface should present a draft for review rather than sending it. Privacy and security also matter because the original email body is transmitted to an external model service. The user must understand this data path and avoid submitting sensitive information unless the deployment and service terms permit it.

1.2 Identification of Problem

The project tackles the friction between receiving an email and composing a suitable reply. Existing mail clients provide composition tools but do not necessarily create a context-aware reply in the recipient's chosen tone. Copying message text into a separate AI tool introduces context switching and manual transfer. A browser extension can reduce those steps, while a simple web interface can support stand-alone use and demonstrate the backend API.

The implementation also exposes engineering problems that must be addressed before reliable use. The supplied extension reads Gmail through page selectors that may change. It calls a local HTTP endpoint, so it is not ready for remote deployment. The backend accepts the request and calls Gemini, but has no explicit validation layer, rate limiting, authentication, or structured error response. The reply parser assumes a particular response shape. These gaps affect reliability, safe handling of user data, and deployment readiness.

1.3 Identification of Tasks

1.3.1 Functional tasks

- Accept the source email text from the React client or Gmail page.
- Allow the web user to select Professional, Casual, Friendly, or no explicit tone.
- Reject empty web-interface input and communicate loading or failure state.
- Send an HTTP POST request to the Spring Boot API with emailContent and tone fields.

- Construct a model prompt and request a generated reply from Gemini.
- Return the reply text for display, copying, or insertion into the Gmail compose editor.

1.3.2 Engineering and documentation tasks

The engineering work includes identifying the boundaries between clients, API controller, service, and external model; documenting the request contract; reviewing project configuration; identifying operational and privacy constraints; and preparing a validation matrix for future execution. The report also distinguishes implemented behavior from recommended work so that proposed functions are not confused with verified capabilities.

1.4 Timeline

The repository does not contain dated milestone records. The following phase plan is therefore a recommended project sequence rather than a claimed historical schedule.

Phase	Work	Completion evidence
1. Requirements	Define inputs, outputs, tones, and review responsibility	Approved requirement list
2. API foundation	Create request model, controller, Gemini service, and configuration	Local API responds to a valid request
3. Client interfaces	Build React form and Gmail content-script action	Reply displayed or inserted in a test mailbox
4. Validation	Test normal, invalid, error, and edge cases	Recorded test results and resolved defects
5. Hardening	Protect secrets, add HTTPS, controls, and operational monitoring	Deployment checklist and security review

1.5 Organization of the Report

Chapter 1 introduces the need, problem, tasks, and planned sequence. Chapter 2 reviews the project context, proposed solution, technology landscape, and objectives. Chapter 3 explains requirements, constraints, architecture, interface behavior, and implementation method. Chapter 4 records the implementation visible in the supplied source and defines validation cases without claiming they have been executed. Chapter 5 summarizes the work and sets out future improvements.

CHAPTER 2

BACKGROUND STUDY

2.1 Timeline of the Reported Problem

Email has developed from a store-and-forward messaging protocol into a standard workspace for task requests, approvals, coordination, and customer communication. As the volume and variety of messages increase, users face repeated drafting decisions: what information to include, how formal to sound, and how to keep the response concise. Modern language models make it possible to generate a candidate response from a supplied text prompt, but the result still depends on the quality of the input and requires human review.

The project follows this progression: routine email creates repeated writing work; language generation can propose a draft; an API separates the client from model access; and a browser integration reduces the steps required to use the service. The supplied code implements a basic version of this path with a React client, Spring Boot service, Gemini call, and Gmail content script. It does not include a research dataset or a measured study of time saved.

2.2 Proposed Solution

SmartMail Assistant provides two client paths to a shared generation service. The React application presents a multiline message field, an optional tone selector, a Generate Reply action, a result panel, and a copy control. The Chrome content script adds an AI Reply button to a Gmail compose toolbar, extracts visible message text, calls the same API route, and inserts returned text into the compose editor. Both clients depend on the Spring Boot service, which builds the prompt and forwards it to the Gemini API.

The user remains in control. A generated reply is displayed or inserted as editable draft text; the project source does not send the email. This workflow is summarized in Figure 2 and the interaction sequence in Figure 3.

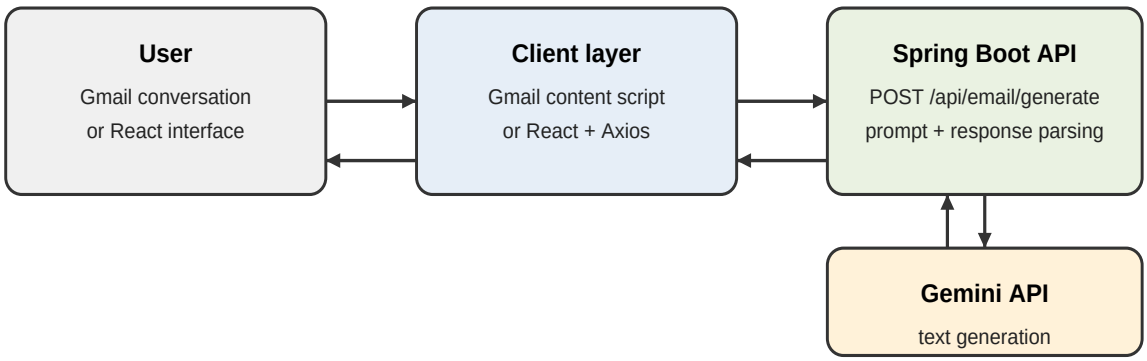


Fig. 2. High-level system architecture

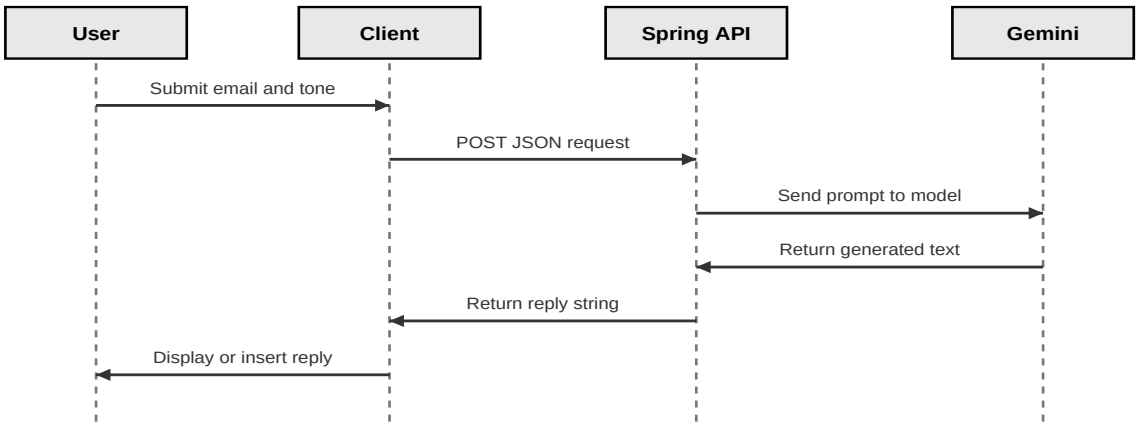


Fig. 3. Request and response sequence

2.3 Bibliometric Analysis

A quantitative bibliometric study requires a defined publication database, search query, date range, screening rules, and reproducible counts. Those materials were not included with the project source, so this report does not invent publication totals or growth percentages. Instead, this section maps the design to the documented technology areas that directly support the implementation: component-based interface development, REST service construction, browser content scripts, and generative text APIs.

Area reviewed	Relevance to this project	Design implication
Component-based UI	React state and event handling underpin the input, loading, result, and error views [1].	Keep user input and generated output visible and editable.
Backend service	Spring Boot supports the server-side application and web endpoint structure [2].	Keep API key use and model integration on the server.
Browser integration	Manifest V3 defines extension metadata and permissions; content scripts interact with matching pages [3].	Limit host access and expect Gmail DOM changes.
Generative API	Gemini provides text generation through a request/response API [4].	Validate requests and handle model errors and output uncertainty.
UI component system	Material UI supplies React form and feedback components [5].	Use accessible labels and clear generation status.

This thematic review informs implementation choices but is not a systematic literature review. Future academic evaluation should define database searches and screening criteria, then compare the system with baseline email drafting using task completion time, user-rated usefulness, correction rate, and privacy comprehension.

2.4 Review Summary

The background review supports a small, server-mediated assistant with a human review step. React provides a state-driven interface; Spring Boot hosts the API; the Chrome extension connects that API to Gmail; and Gemini supplies candidate language. These technologies solve different parts of the flow and do not by themselves guarantee correctness, privacy, or availability. The system's quality depends on validation at every boundary: user input, API request, external model call, returned response, and insertion into a draft.

2.5 Problem Definition

Users spend avoidable effort composing routine email replies and may need to switch between their inbox and a separate drafting tool. The project problem is to provide a convenient mechanism that uses the source message and a requested tone to produce a reviewable reply draft, while preserving the user's decision to edit, copy, insert, or discard the result.

The engineering problem is to do so reliably and safely across a browser client, a backend service, and an external model API. The solution must handle empty or malformed input, service failures, response variations, secret management, cross-origin or extension access, and the privacy implications of transmitting email text.

2.6 Goals / Objectives

- Provide a simple input path for email text and optional tone.
- Generate a reply through a server-side model integration without exposing the API secret in the client.
- Make the result easy to review and copy, and insert it into Gmail where the extension is used.
- Communicate request progress and recoverable errors in the user interface.
- Define a testable API contract and validation cases.
- Identify the changes needed for privacy, security, accessibility, and production deployment.

2.7 Why Use These Technologies Over Other Options

Technology	Reason for selection in this project	Trade-off
React	Fits an interactive form and supports local state for input, loading, errors, and result [1].	Requires a JavaScript build/runtime and disciplined component design.
Material UI	Provides ready-made form controls and layout components already used by the interface [5].	Adds dependency weight and requires accessibility review.
Spring Boot / Java	Provides a conventional REST service and server-side configuration for the model call [2].	The current service blocks on the external response and has limited failure handling.
Chrome Manifest V3	Allows an extension content script to augment Gmail pages under declared host access [3].	Gmail markup is not a stable public integration API.
Gemini API	Supplies text generation without hosting a language model locally [4].	External availability, cost, quotas, policy, and data handling apply.

The selected stack is visible in the project manifests and source files. The selection is appropriate for a prototype that demonstrates client-to-server-to-model integration; it does not establish that these technologies are the only or optimal choices for every deployment.

CHAPTER 3

DESIGN AND PROCESS

3.1 Evaluation and Selection of Specifications / Features

The core specification is intentionally narrow: accept a message and tone, generate one reply, and return it for user review. The feature set below separates code-backed behavior from requirements that remain incomplete or unverified.

Feature	Implementation evidence	Status
Email text entry	React multiline TextField and Gmail DOM extraction	Implemented in source
Tone selection	React offers Professional, Casual, Friendly; extension sends professional	Partial: client behaviors differ
Prompt construction	Service adds a reply instruction, optional tone, and original message	Implemented; tone wording may conflict
Model request	WebClient posts JSON to configured Gemini URL	Implemented in source; live call not verified
Result handling	React result panel and copy; extension inserts into compose editor	Implemented in source; Gmail behavior not verified
Validation and resilience	Web form rejects blank text; backend parser assumes response fields	Partial; server-side safeguards absent
Privacy/security	API key supplied through server configuration; no storage code observed	Partial; transport and access controls need hardening

3.2 Design Constraints

- External model dependency: generation requires network access, a valid credential, an available model endpoint, and sufficient quota.
- Sensitive input: email text can contain personal or confidential information; the service forwards it to an external provider and the current code has no consent or redaction step.
- Credential handling: the service constructs the API URL with a key query parameter. Production deployment should follow current provider guidance and avoid exposing credentials in logs, source control, browser code, or URLs.
- Local endpoint: the React app and extension point to http://localhost:8080, which is suitable only for a local development setup and does not provide encrypted transport.
- Browser page dependence: Gmail class names and editor structure can change; the selectors need monitoring and resilient lookup behavior.
- Prompt consistency: the base prompt asks for a professional reply even when a different tone is selected, which can produce conflicting instructions.
- Response variability: generated text can vary, be empty, or arrive in an unexpected response shape; the parser and UI should handle these cases explicitly.

- No persistent account or history subsystem: the supplied source has no user authentication, stored drafts, analytics, or audit history.

3.3 Design Flow

The user workflow begins with message capture and ends when the user reviews and chooses how to use the draft. The generation request travels from a client to the backend and then to Gemini; the returned text follows the reverse path. Figure 4 summarizes this workflow. Figure 5 separates the client, controller, service, and external model responsibilities.

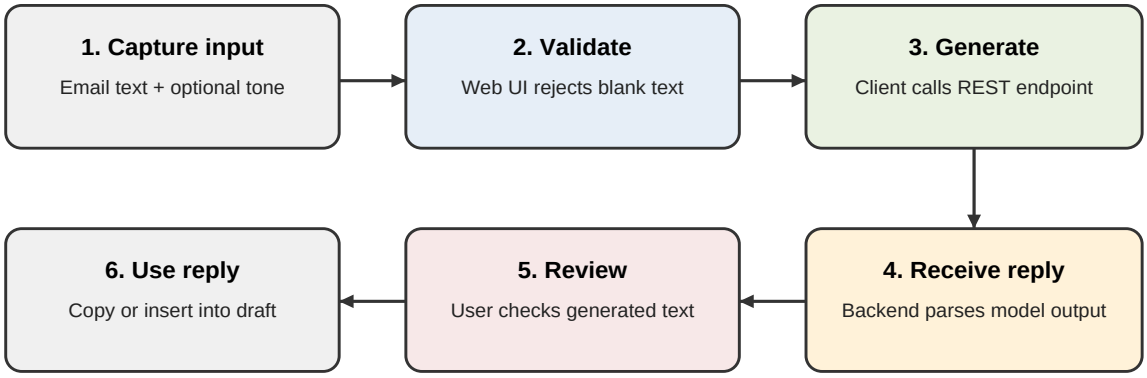


Fig. 4. User workflow for generating a reply

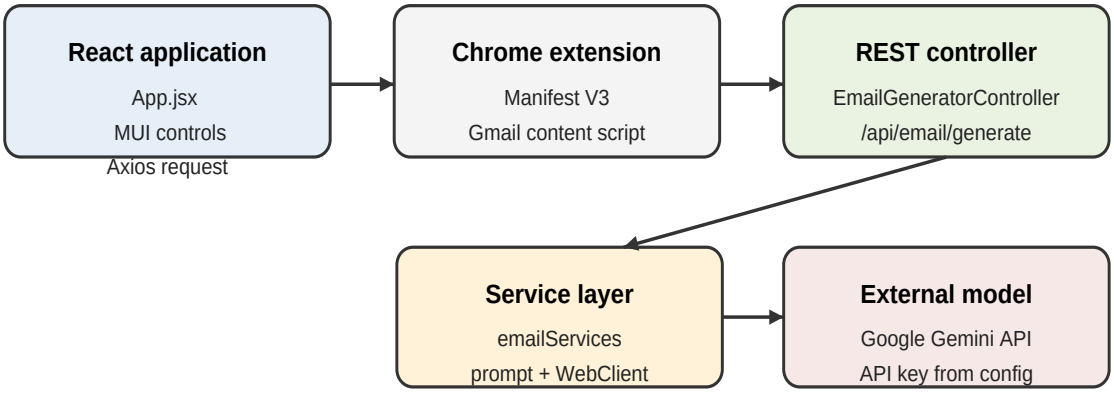


Fig. 5. Software component view

3.4 Creation of SmartMail Assistant

3.4.1 React interface

The React application stores emailContent, tone, generatedReply, loading, and error in component state. The user enters text in a multiline field and can select one of three named tones or leave tone unset. The Generate Reply button is disabled while the text is blank or a request is in progress. On submission, Axios posts JSON to http://localhost:8080/api/email/generate. On success, the returned string is displayed in a result panel; the Copy to Clipboard control uses the browser clipboard API. On failure, a generic retry message appears. The source uses Material UI components for layout, text entry, selection, button, and progress feedback.

The form-level empty-input check improves usability, but equivalent server-side validation is necessary because clients can be bypassed. The current interface also treats successful response data as text or serializes it if it is not a string; a documented response schema would reduce ambiguity.

3.4.2 Spring Boot API

EmailGeneratorController is mapped under /api/email and accepts POST /generate. It binds a JSON EmailRequest containing emailContent and tone, delegates to emailServices.generateReply, and returns the resulting string in an HTTP 200 response. Cross-origin access is currently configured for all origins. The service uses WebClient to submit a JSON contents/parts/text payload to the configured Gemini URL and then parses candidates[0].content.parts[0].text from the response.

The request contract is compact and easy to integrate, but the current controller does not validate null or oversized content, define a typed response object, or translate upstream failures into suitable HTTP statuses. The service uses block() to wait for the model result. Error handling catches parsing exceptions and returns an error string, which makes success and failure difficult for the client to distinguish.

3.4.3 Gmail extension

The Manifest V3 manifest grants activeTab and storage permissions, declares localhost and mail.google.com host access, and loads content.js and content.css on Gmail pages. The content script observes DOM mutations, searches for a compose toolbar, adds an AI Reply button, extracts message text, calls the backend with a fixed professional tone, and inserts the returned text into a Gmail editable textbox. This is a direct DOM integration, not a Gmail API integration.

The implementation has selector-loop control-flow issues: getEmailContent and findComposeToolbar return from the first selector iteration when the first candidate is absent, so later fallback selectors are not tried. The observer's handling of a newly opened compose window is also dependent on which nodes appear in the mutation batch. These issues should be corrected and tested against representative Gmail layouts before claiming dependable integration.

3.4.4 Request and response contract

Element	Type	Purpose
emailContent	String	Source email text used as generation context.
tone	String (optional)	Style instruction such as Professional, Casual, or Friendly.
Request	JSON object	POST body sent to /api/email/generate.
Response	Text string	Generated reply returned to the client; no subject line requested.

The code does not currently enforce an allow-list of tones or a maximum email length. A production API should validate both fields, define a stable JSON response schema, and avoid returning raw implementation errors to the user.

3.5 Logo Designing and Naming

The project is named SmartMail Assistant, a descriptive title that communicates the combination of email drafting and AI assistance. No project-specific logo or brand asset was found in the supplied source files. The report therefore retains the Chandigarh University mark on the cover, as requested, and does not invent a product logo or claim that logo-design work has been completed.

3.6 Implementation Plan / Methodology

The implementation can be understood as an iterative prototype process: establish the backend endpoint and model call, connect a web client, add Gmail page integration, inspect behavior, and refine the response and error paths. The files show the first three activities in code, but no issue tracker, dated plan, deployment record, or end-to-end test evidence was included. Accordingly, the phases below are presented as a repeatable method for completing and validating the system.

Step	Method	Deliverable
1. Define	Record functional and non-functional requirements; agree what email content may be sent to a model.	Requirements and data-flow note
2. Design	Specify the endpoint, payload, response schema, failure states, and trust boundaries.	API contract and component diagram
3. Implement	Build client, API controller, model service, and Gmail content script in small increments.	Versioned code and configuration
4. Validate	Run unit, API, UI, extension, security, and usability checks on test messages.	Test log with evidence and defects
5. Release	Configure HTTPS, secret storage, access controls, monitoring, and user notice.	Reviewed deployment package

CHAPTER 4

RESULT ANALYSIS AND VALIDATION

4.1 Implementation of Solution

Source inspection confirms a functional path has been implemented at the code level: a client builds an email/tone request, the controller forwards it to a service, the service calls the configured Gemini endpoint, and reply text is returned. The React path displays and copies the result. The Gmail script attempts to add a compose action and insert the result. These statements describe source behavior; no live Gemini response, browser session, benchmark, or user study was supplied as evidence.

The resulting prototype demonstrates the intended architecture but has incomplete validation and deployment support. Table 4.1 summarizes feature coverage and evidence limits.

Area	Observed result in source	Evidence boundary
Web input	React form captures content and optional tone; blank input is rejected.	No rendered or browser execution evidence supplied.
Backend route	POST /api/email/generate binds EmailRequest and returns text.	No running API trace supplied.
Gemini integration	Service builds a prompt and extracts candidate text.	No credential or live provider result supplied.
Gmail action	Content script attempts button injection and draft insertion.	Gmail DOM behavior not exercised in evidence.
Error handling	Web UI shows a generic error; parser returns an error string.	Failure status contract is not defined.
Test coverage	A Spring application-context test class exists in the repository.	No test execution result is reported here.

4.1.1 Report Preparation

The report was prepared from the supplied repository, including the README, React component and package metadata, Spring Boot controller and service, application properties, extension manifest, and Gmail content script. The PDF format provided the chapter sequence and visual pattern. No screenshots, user records, production logs, measured response times, or formal test reports were present in those materials. The report therefore uses architecture diagrams and explicitly identified implementation summaries in place of unsupported screenshots or quantitative results.

4.1.2 Project Management and Communication

The repository contains separate directories for the React application, backend, and browser extension, which makes the principal technical boundaries identifiable. The README documents a local setup concept and API example, but some of its claims and paths do not match the current source layout. For example, it describes React/MUI as the extension UI while the manifest points to a standalone content script and the React app is a separate Vite project. Project handover should reconcile the README with actual directories, configuration, tone behavior, and installation steps.

No meeting notes, work assignments, milestone records, or version-control history analysis were supplied as project evidence. This section does not attribute team actions or dates that cannot be verified.

4.1.3 Testing / Characterization / Interpretation / Data Validation

The following matrix is a proposed validation plan derived from the implementation. It records expected behavior and is not a claim that tests have been run. Each case should be executed with a non-sensitive test message after the endpoint and extension are configured.

ID	Scenario	Expected result	Status in this report
V1	Valid email with Professional tone	API returns a non-empty reply draft.	Not executed
V2	Valid email with Casual/Friendly tone	Reply follows selected tone without conflicting default instruction.	Not executed
V3	Blank or whitespace-only request to API	Client/server rejects it with a clear validation response.	Not executed; server validation absent
V4	Gemini timeout, quota error, or invalid key	Client receives a meaningful failure status; secret is not exposed.	Not executed; upstream mapping incomplete
V5	Response lacks candidates or text part	Service returns a controlled error, not an uncaught exception.	Not executed; parser assumes shape
V6	Gmail page lacks first toolbar selector	Fallback selectors are tried and no duplicate button is created.	Not executed; loop returns early
V7	Long email input	Length limit and user feedback are applied before model call.	Not executed; limit absent
V8	Check browser network and server logs	No API key is exposed; transport is encrypted in deployment.	Not executed; local HTTP configured

Interpretation: implementation coverage is strongest for the happy-path sequence and the web interface's local user feedback. Validation coverage, privacy controls, upstream failure handling, Gmail selector resilience, and deployment hardening remain the priority areas. No performance or accuracy metric is reported because no measurements were supplied or collected.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

SmartMail Assistant demonstrates a basic AI-assisted email reply workflow using a React and Material UI front end, a Spring Boot REST API, a Gemini model call, and a Chrome content script for Gmail. The design keeps generation behind a backend service and returns a draft for the user to review. The project therefore establishes a clear prototype architecture for reducing repetitive reply-writing effort.

The supplied implementation does not yet establish reliable production behavior. The browser and web clients have differing tone capabilities; Gmail integration relies on brittle selectors; the backend lacks robust request validation and upstream error mapping; the model response parser assumes a fixed shape; and the configured local HTTP endpoint is not suitable for deployment. Email privacy, credential handling, accessibility, and testing also need explicit treatment. These findings define the next engineering steps rather than diminishing the prototype's value as an integration demonstration.

5.2 Future Work

5.2.1 Improve prompt and response handling

Build the prompt from a neutral instruction plus exactly one validated tone choice. Request concise output without an invented subject line, and consider a structured response containing reply text and safe status metadata. Parse missing or blocked model output explicitly, set timeouts, and translate provider failures into stable HTTP error responses.

5.2.2 Strengthen privacy and security

Use HTTPS for all non-local traffic, keep API credentials in a secret manager or environment configuration, and follow the model provider's current authentication guidance. Add input-size limits, rate limiting, abuse controls, origin restrictions, and a user-facing explanation of where message content is processed. Avoid logging message content or secrets. Review the data-retention and processing terms of the selected model service before deployment.

5.2.3 Make the Gmail integration more reliable

Correct selector iteration so all fallbacks are tried, scope message and compose elements to the active conversation, prevent duplicate controls, and handle Gmail navigation changes. Prefer supported extension and mail APIs where the required functionality permits. Add compatibility checks and a graceful fallback that lets the user copy the response if insertion fails.

5.2.4 Expand validation

Add service-level tests for prompt construction and response parsing, controller tests for validation and status codes, and integration tests using a mocked model response. Exercise the extension against test Gmail layouts, including missing toolbar elements and multiple compose windows. Conduct usability checks for tone appropriateness, edit effort, and user understanding of the model's limitations. Report measured results with sample size and test conditions.

5.2.5 Complete deployment and accessibility work

Reconcile the README with the actual directory structure, provide a reproducible setup, configure a production API URL and deployment profile, and monitor availability without collecting message content. Review keyboard access, focus order, accessible labels, progress announcements, and error descriptions in both client paths. Future extensions may include user-defined tone guidance, language selection, and draft refinement, provided they do not obscure the user's review and control.

REFERENCES

[1] React Team, *Quick Start*, React documentation. Available: https://react.dev/learn (accessed 29 October 2026).

[2] VMware, Inc., *Spring Boot Reference Documentation*. Available: https://docs.spring.io/spring-boot/reference/ (accessed 29 October 2026).

[3] Chrome for Developers, *Manifest file format*. Available: https://developer.chrome.com/docs/extensions/reference/manifest (accessed 29 October 2026).

[4] Google AI for Developers, *Gemini API: Generating content*. Available: https://ai.google.dev/api/generate-content (accessed 29 October 2026).

[5] MUI, *Material UI documentation*. Available: https://mui.com/material-ui/getting-started/ (accessed 29 October 2026).

[6] R. Soni, *SmartMail Assistant project source repository*, supplied project files: README, React interface, Spring Boot backend, and Chrome extension. Reviewed 29 October 2026.

PLAGIARISM REPORT

Attach the institution-issued similarity report here after the report has been processed through the approved Chandigarh University system.